

ExoHabitAI Backend: Technical Architecture & API Specification

1. Project Overview

ExoHabitAI is a machine learning-powered REST API built with Flask that predicts whether a given exoplanet is potentially habitable for human life based on a set of astrophysical and environmental features. The system accepts scientific measurements about a planet and its host star, runs them through a trained Logistic Regression model, and returns a habitability prediction along with a confidence score.

1.1 System Architecture

The backend consists of two main Python files:

File	Role	Key Responsibilities
app.py	Flask Application Entry Point	Route definitions, request handling, response formatting
utils.py	Utility & Helper Module	Model loading, feature definitions, input validation

1.2 Technology Stack

Technology	Version	Purpose
Flask	2.3.2	Web framework for building REST API endpoints
Flask-CORS	4.0.0	Cross-Origin Resource Sharing for frontend connection
scikit-learn	1.3.0	Machine learning — Logistic Regression classifier
pandas	2.0.3	Data manipulation and DataFrame operations
numpy	1.24.3	Numerical computing support
joblib	1.3.1	Model serialization (.pkl file loading)

2. Theory & Scientific Background

2.1 Exoplanet Habitability

An exoplanet is a planet that orbits a star outside our Solar System. Determining whether such a planet could support life requires analyzing a combination of planetary properties and host star characteristics. The Habitable Zone (HZ) — sometimes called the Goldilocks Zone — is the range of orbital distances from a star where liquid water could theoretically exist on a planet's surface.

Key factors that influence habitability include:

- Planetary equilibrium temperature (`pl_eqt`) — must fall within a narrow range (~200–320 K) for liquid water
- Orbital semi-major axis (`pl_orbsmax`) — determines distance from host star
- Planet radius and mass — affects gravitational retention of an atmosphere
- Host star spectral type — different star types (F, G, K, M) emit different radiation levels
- Stellar luminosity and metallicity — affects stellar evolution and planetary formation

2.2 Machine Learning Model — Logistic Regression

The prediction engine is a Logistic Regression model, a statistical classification algorithm that models the probability of a binary outcome using the logistic (sigmoid) function. Rather than building multiple trees, it finds a single optimal decision boundary in the feature space. Logistic Regression is well-suited for this task because:

- It outputs probability estimates natively via `predict_proba()`, enabling confidence scoring
- It is interpretable — coefficients show which features most influence habitability
- It trains efficiently on tabular scientific data with many numeric features
- It generalizes well when features are scaled and the decision boundary is roughly linear

2.3 Confidence Threshold Logic

The API applies a strict confidence threshold of 0.90 (90%). If the model's predicted probability for a planet being habitable is below this threshold, the prediction is automatically overridden to 0 (Not Habitable). The confidence score is then inverted to reflect confidence in the non-habitable classification. This design minimizes false positives — the system only reports a planet as habitable when it is highly confident.

Confidence Override Rule
IF probability < 0.90 → prediction = 0 (Not Habitable)
→ confidence = 1 - probability (inverted)
IF probability >= 0.90 → prediction = 1 (Habitable)
→ confidence = original probability

2.4 Feature Engineering

Beyond raw measurements, the model uses several engineered features that combine multiple properties into single scores:

Engineered Feature	Description
temp_score	Normalized score for planetary equilibrium temperature suitability
dist_score	Score based on orbital distance relative to habitable zone
lum_score	Score reflecting how star luminosity supports habitability
stellar_temp_score	Compatibility of stellar temperature with life-supporting conditions
stellar_mass_score	Star mass suitability for stable, long-lived stellar system
stellar_compatibility_index	Composite index combining multiple stellar factors
orbital_stability	Metric assessing whether the orbital path is stable long-term

3. Input Features Reference

The model requires exactly 22 input features. These are divided into Planetary Features, Stellar Features, Spectral Type Flags, and Engineered Scores.

3.1 Planetary Features

Feature Name	Type	Description
pl_rade	Numeric (float)	Planet radius in Earth radii
pl_bmasse	Numeric (float)	Planet mass in Earth masses
pl_orbper	Numeric (float)	Orbital period in days
pl_orbsmax	Numeric (float)	Orbital semi-major axis in AU (Astronomical Units)
pl_eqt	Numeric (float)	Planetary equilibrium temperature in Kelvin

3.2 Stellar Features

Feature Name	Type	Description
st_teff	Numeric (float)	Host star effective temperature in Kelvin
st_rad	Numeric (float)	Host star radius in solar radii
st_mass	Numeric (float)	Host star mass in solar masses
st_met	Numeric (float)	Host star metallicity [Fe/H] in dex
st_lum	Numeric (float)	Host star luminosity in log solar luminosity

3.3 Spectral Type Flags (Binary)

Exactly one of the five spectral type flags must be set to 1. All others must be 0.

Feature Name	Type	Description
spec_F	Binary (0 or 1)	F-type star (hotter, more luminous than Sun)
spec_G	Binary (0 or 1)	G-type star (Sun-like, ideal for habitability)

spec_K	Binary (0 or 1)	K-type star (slightly cooler than Sun)
spec_M	Binary (0 or 1)	M-type star (red dwarf, most common in galaxy)
spec_m	Binary (0 or 1)	Sub-M type or unclassified dwarf star

3.4 Engineered Score Features

Feature Name	Type	Description
temp_score	Numeric (float)	Computed temperature suitability score
dist_score	Numeric (float)	Computed orbital distance suitability score
lum_score	Numeric (float)	Computed stellar luminosity suitability score
stellar_temp_score	Numeric (float)	Stellar temperature compatibility score
stellar_mass_score	Numeric (float)	Stellar mass compatibility score
stellar_compatibility_index	Numeric (float)	Combined stellar habitability index
orbital_stability	Numeric (float)	Orbital stability metric

4. API Endpoints

The ExoHabitAI backend exposes three REST API endpoints. All endpoints return JSON responses.

Base URL: http://localhost:5000

Protocol: HTTP/HTTPS

Data Format: JSON (application/json)

CORS: Enabled for all origins

4.1 GET / — Health Check

A simple health check endpoint to verify the API server is running.

Property	Value
Method	GET
URL	/
Authentication	None required
Request Body	None

Request Example

```
GET http://localhost:5000/
```

Success Response — 200 OK

```
{
  "status": "success",
  "message": "ExoHabitAI Backend Running"
}
```

4.2 POST /predict — Habitability Prediction

The primary endpoint. Accepts a JSON body containing all 22 required planetary and stellar features, runs them through the trained Logistic Regression model (logistic_pipeline.pkl), and returns a habitability prediction with an optional confidence score.

Property	Value
Method	POST
URL	/predict
Content-Type	application/json
Authentication	None required
Request Body	JSON object with all 22 feature fields

Request Format

Send a POST request with a JSON body containing all 22 features listed in Section 3. All numeric features must be valid float values. Binary features must be 0 or 1.

Request Example

```
POST http://localhost:5000/predict
Content-Type: application/json

{
  "pl_rade": 1.24,
  "pl_bmasse": 2.10,
  "pl_orbper": 365.25,
  "pl_orbsmax": 1.02,
  "pl_eqt": 278.5,
  "st_teff": 5778.0,
  "st_rad": 1.0,
  "st_mass": 1.0,
  "st_met": 0.01,
  "spec_F": 0,
  "spec_G": 1,
  "spec_K": 0,
  "spec_M": 0,
  "spec_m": 0,
  "st_lum": 0.02,
  "temp_score": 0.92,
  "dist_score": 0.88,
  "lum_score": 0.85,
  "stellar_temp_score": 0.90,
  "stellar_mass_score": 0.87,
  "stellar_compatibility_index": 0.89,
  "orbital_stability": 0.95
}
```

Success Response — 200 OK

```
{
  "status": "success",
  "prediction_result": 1,
  "confidence_score": 0.9432
}
```

Response Fields

Field	Type	Description
status	String	"success" if request was processed correctly
prediction_result	Integer (0 or 1)	1 = Habitable, 0 = Not Habitable
confidence_score	Float or null	Probability score (0.0–1.0). null if model does not support predict_proba()

Error Responses

HTTP Code	Cause	Response Message Example
400 Bad Request	Missing JSON body	"No JSON data"
400 Bad Request	Missing feature field	"Missing: pl_rade"
400 Bad Request	Invalid numeric value	"Invalid value: st_teff"
400 Bad Request	Invalid binary value	"spec_G must be 0 or 1"
400 Bad Request	Null feature value	"pl_eqt is null"
500 Internal Error	Model not loaded	"ML Model is not loaded"

4.3 GET /rank — Ranked Exoplanet List

Returns a ranked list of exoplanets from a pre-computed dataset. The list is sorted by the Rank column (ascending) and the number of results can be controlled using a query parameter.

Property	Value
Method	GET
URL	/rank
Authentication	None required
Request Body	None
Query Parameter	limit (integer, optional, default = 10)
Data Source	data/habitability_ranked.csv (loaded at startup)

Request Example — Default (Top 10)

```
GET http://localhost:5000/rank
```

Request Example — Custom Limit

```
GET http://localhost:5000/rank?limit=5
```

Success Response — 200 OK

```
{
  "status": "success",
  "returned_count": 5,
  "data": [
    {
      "Rank": 1,
      "pl_name": "Kepler-442b",
      "pl_rade": 1.34,
      "pl_eqt": 233.0,
      "...", // all columns from CSV
    },
    { ... }
  ]
}
```

Response Fields

Field	Type	Description
status	String	"success" if data loaded and returned correctly
returned_count	Integer	Number of planet records returned (equals limit or available count)
data	Array of Objects	Array of planet records from ranked dataset. Each object contains all CSV columns. Null values replace NaN entries.

Error Responses

HTTP Code	Cause	Response Message
500 Internal Error	CSV file missing or failed to load at startup	"Ranked data is not available"
500 Internal Error	CSV does not contain a Rank column	"Rank column missing in dataset"

5. Input Validation Logic

All incoming requests to the /predict endpoint are validated by the `validate_input()` function in `utils.py` before the model is called. Validation happens in four sequential steps:

Step 1 — JSON Presence Check

The function first verifies that a JSON body was received. If `request.get_json()` returns `None`, a 400 error is returned immediately.

Step 2 — Feature Completeness Check

Every feature in the FEATURES list (22 total) is checked for presence in the incoming JSON. If any feature key is missing, an error is returned identifying the missing field.

Step 3 — Numeric Feature Validation

For the 17 numeric features, validation checks that the value is not None and can be successfully converted to float. A failed float() conversion returns a descriptive error.

Step 4 — Binary Feature Validation

For the 5 spectral type flags (spec_F, spec_G, spec_K, spec_M, spec_m), the value must be one of: 0, 1, '0', '1', True, or False. Any other value returns a validation error.

Validation Summary	
Total Features Validated: 22	
Numeric Features:	17 (must be valid float, not None)
Binary Features:	5 (must be 0 or 1)
Validation Order:	JSON → Completeness → Numeric → Binary
On Failure:	Returns 400 Bad Request with descriptive message

6. Project File Structure

```
project-root/
├─ app.py           # Flask application, routes, endpoints
├─ utils.py         # FEATURES list, load_model(),
validate_input()
├─ requirements.txt  # Python dependencies
├─ models/
│   └─ logistic_pipeline.pkl  # Trained scikit-learn model (joblib
serialized)
└─ data/
    └─ habitability_ranked.csv # Pre-ranked exoplanet dataset for /rank
endpoint
```

6.1 Model File Location

The model is loaded at server startup from `models/logistic_pipeline.pkl` (path is resolved relative to `utils.py` using Python's `pathlib`). If the file is not found, the model variable is set to `None` and `/predict` will return a 500 error.

6.2 Ranked CSV Location

The ranked CSV is loaded at startup from `data/habitability_ranked.csv` (path resolved relative to `app.py`). If the file cannot be loaded, `ranked_df` is set to an empty `DataFrame` and `/rank` will return a 500 error.

7. Complete Error Reference

Endpoint	HTTP Code	Error Condition
<code>/predict</code>	400	No JSON data in request body
<code>/predict</code>	400	One or more required features are missing
<code>/predict</code>	400	A numeric feature has null value
<code>/predict</code>	400	A numeric feature cannot be converted to float
<code>/predict</code>	400	A binary feature is not 0 or 1
<code>/predict</code>	500	Model file not found or failed to load
<code>/rank</code>	500	Ranked CSV not found or failed to load at startup
<code>/rank</code>	500	CSV loaded but 'Rank' column not present

8. Running the Server

8.1 Install Dependencies

```
pip install -r requirements.txt
```

8.2 Start the Flask Server

```
python app.py
```

The server will start on port 5000 with debug mode enabled (development only). You should see:

```
INFO:root:Model loaded
INFO:root:Ranked CSV loaded successfully
* Running on http://127.0.0.1:5000
```